COMPTE RENDU AOP DRONES
Semestre 3
Modélisation d'un drones Anafi (Bebop 2)



NOM : Kouthieye Thiaw
Groupe : B1

Table des matières

I) Introduction 3
II) présentation du projet 3
III) L'hyperealité 3
IV) Modelisation du drone suite a un essai indicielle 4
A) Modelisation du pitch 4
1) Rappels theoriques 5
2) Modele Matlab 5
3) Calcul du PID 6
B) Modélisation du roll 7
1) Identification 7
2) Calcul du PID 9
V) Protocole UDP et TCP/IP 9
A) Le Serveur UDP 9
B) Le Serveur TCP 10
VI) Proposition et idées d'amélioration de l'hyperréalité 11
A) Intégration de la Physique Virtuelle dans le Monde Réel 11
B) Immersion et Retour de Force 11
C) Compensation Prédictive par Intelligence Artificielle 11
VII) Conclusion 11

I) Introduction

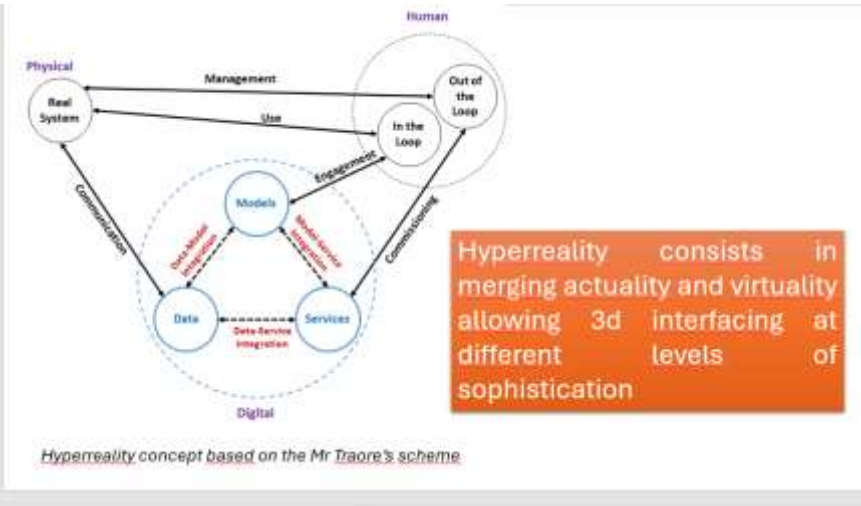
Ce projet, réalisé dans le cadre de l'AOP du semestre 3, se concentre sur l'analyse dynamique et le pilotage du drone Parrot Bebop 2. L'étude a débuté par une phase de modélisation théorique visant à définir les équations physiques régissant le comportement du vecteur aérien. Dans un second temps, nous avons exploité le concept d'hyperréalité via l'environnement de supervision développé en Visual Basic par M. Mathieu, permettant d'interagir directement avec le jumeau numérique 3D du drone pour valider les lois de commande. Enfin, le projet s'est attaché à décrypter l'architecture informatique interne et les protocoles de communication qui assurent le lien entre le logiciel de contrôle et le robot physique.

II) Présentation du projet

Ce projet s'inscrit dans une transition technologique du **Parrot Bebop 2** vers le **Parrot Anafi**, un drone plus récent et performant. L'objectif central est la mise en œuvre de l'**Hyperréalité**, un concept fusionnant le monde physique (le drone en vol) et le monde numérique (Jumeau Numérique / Simulation) pour permettre un contrôle avancé et une interaction immersive.
Le système ne se limite pas au pilotage classique ; il intègre des interfaces innovantes (capteur EMG Myo pour le contrôle gestuel) et des boucles d'asservissement basées sur l'identification mathématique rigoureuse du système.

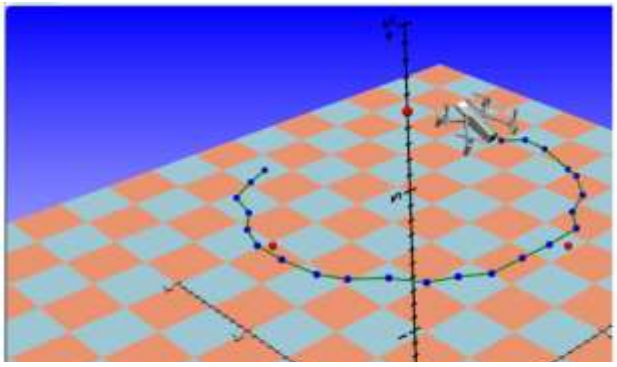
III) L'hyperealité

L'hyperréalité dans ce projet ne se limite pas à une simple visualisation 3D. Elle repose sur une **synchronisation bidirectionnelle** permanente entre le drone physique (Parrot Anafi) et son jumeau numérique. Contrairement à une simulation classique où tout est virtuel, ici, le réel alimente le virtuel, et le virtuel contraint le réel. Cette architecture permet de sécuriser les vols et de valider des lois de commande complexes sans risquer l'intégrité de l'appareil.



Hyperreality concept based on the Mr Traore's scheme

L'hyperréalité ne serait qu'une coquille vide sans un modèle physique réaliste. C'est ici qu'intervient le travail d'identification réalisé sous MATLAB . En analysant la réponse du drone à des commandes brutes (fichiers CSV), on aextrait sa **fonction de transfert**.
Ce modèle mathématique est le "cerveau" du jumeau numérique. Il permet au système de **prédire** le comportement du drone. Lorsqu'un utilisateur envoie une commande (par exemple via le brassard Myo), celle-ci traverse d'abord les algorithmes de régulation (PID) basés sur ce modèle virtuel. Le système "lisse" et stabilise l'ordre avant de l'envoyer aux moteurs du drone réel. Le virtuel agit donc comme un filtre intelligent qui améliore les performances du réel.





IV) Modélisation du drone suite à un essai indicielle

A) Modélisation du pitch

La stratégie d'acquisition des données repose sur un protocole d'essai indiciel en boucle ouverte, méthode standard pour la caractérisation dynamique d'un système complexe. Le drone, initialement maintenu en vol stationnaire stabilisé, est soumis à une excitation instantanée sous la forme d'un échelon de commande d'une amplitude de 5%. Durant cette phase critique, le logiciel de supervision enregistre à haute fréquence la réponse temporelle du vecteur aérien via ses capteurs inertiels embarqués. Cette procédure permet d'isoler la réponse transitoire du système, caractérisant ainsi la manière dont le drone rallie sa nouvelle consigne et révélant ses propriétés

intrinsèques d'inertie et d'amortissement, indépendamment des boucles de correction de position. À l'issue de cette campagne expérimentale, les données brutes sont exportées au format CSV puis importées dans l'environnement MATLAB. Ce traitement post-vol est indispensable pour procéder à l'identification mathématique du système, simuler son comportement théorique et en déduire les paramètres caractéristiques nécessaires à la synthèse du correcteur.

1) Rappels théoriques

L'objectif de cette étude est de traduire le comportement physique du drone en une équation mathématique exploitable. Lorsque nous pilotons le pitch ,nous envoyons une commande d'angle et nous observons une réaction en vitesse longitudinale. Pour l'ordinateur de bord, cette relation de cause à effet se modélise par une **Fonction de Transfert**, notée H(p) ,qui est simplement la division mathématique de la Sortie (la vitesse du drone) par l'Entrée (l'ordre d'inclinaison).

Dans une approche simplifiée, le drone réagit comme un système du premier ordre. Cela signifie qu'il ne change pas de vitesse instantanément à cause de son inertie. Cette dynamique se résume par l'équation standard suivante :

$$H(p) = \frac{K}{1+\tau p} e^{-TP}$$

Cette formule repose sur trois paramètres clés que nous devons trouver. Le premier est le **Gain Statique K**, qui représente la puissance de réaction du drone (si je demande 10 degrés, à quelle vitesse finale va-t-il voler ?). Le second est la **Constante de Temps τ**, qui mesure la nervosité de l'appareil, c'est-à-dire le temps qu'il met pour effectuer la majorité de son accélération. Enfin, le terme exponentiel représente le **Retard Pur T**, c'est-à-dire le petit délai incompressible entre l'envoi de l'ordre en Wi-Fi et le début de la rotation des moteurs.

Cependant, pour atteindre le niveau de précision requis par l'hyperréalité, nous ne nous sommes pas limités à ce modèle théorique simple. Nous avons utilisé une sous MATLAB, qui cherche une équation plus complexe (d'un ordre plus élevé) pour coller parfaitement à la réalité des données enregistrées. Enfin, comme le drone est piloté par un processeur numérique qui calcule pas à pas (toutes les 0,07 secondes), nous avons converti cette équation continue en un modèle discret (transformée en Z), seul format compréhensible par le programme de vol.

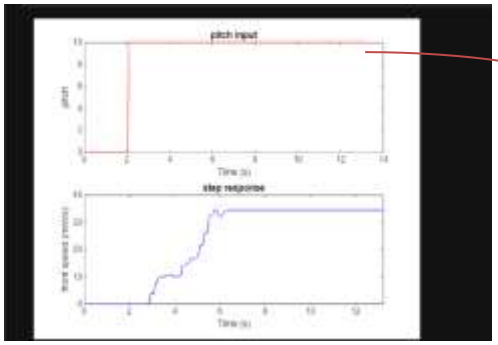
2) Modele Matlab

Après avoir importé le fichier CSV obtenu lors de l'essai , on a le code Matlab ci-dessous qui représente la modélisation de notre modèle (pitch)

```
% display experimental results
signal1 =readtable('inputpitch10extended.csv', 'Delimiter', ';', 'NumHeaderLines', 1);
signal2 =readtable('outputpitch10extended.csv', 'Delimiter', ';', 'NumHeaderLines', 1);

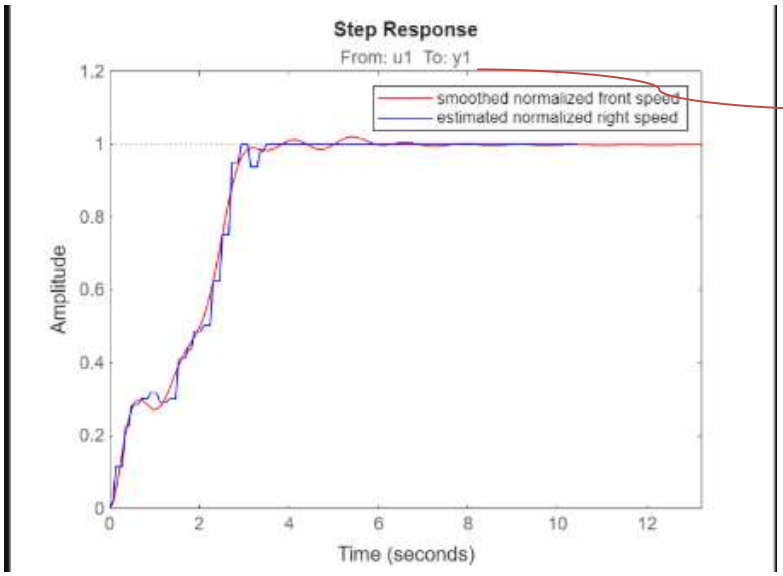
signal1 = signal1{:,:}; % This creates a matrix
signal2 = signal2{:,:}; % This creates a matrix
% vector = T{1, :};
t1=signal1(:,1);
pitch=signal1(:,2);
t2=signal2(:,1);
frontspeed=signal2(:,2);
indices = find(t2 < 2.87);
frontspeed2=frontspeed(:);
frontspeed2(indices)=0;

subplot(2,1,1);
plot(t1,pitch,Color="red"); hold on
xlabel("Time (s)")
ylabel("pitch")
title("pitch input")
subplot(2,1,2);
% plot(t2,frontspeed,Color="red"); hold on
plot(t2,frontspeed2,Color="blue"); hold on
xlabel("Time (s)")
ylabel("front speed (mm/s)")
title("step response")
xlim([min(t1) max(t1)])
```



Réponse obtenu suite avec le passage de 0 q un échelon de 10

Le passage des données expérimentales à la fonction de transfert s'effectue via l'algorithme d'identification numérique tfest de MATLAB, qui procède par minimisation de l'erreur de prédiction entre la réponse réelle mesurée (nettoyée de son offset initial) et un modèle mathématique théorique. Contrairement à une approche simplifiée du premier ordre, votre script impose une structure de type "Boîte Noire" d'ordre élevé, caractérisée par **6 pôles et 4 zéros**, ce qui permet de capturer les modes vibratoires complexes et l'inertie fine du drone. Les caractéristiques physiques figées dans ce modèle sont un **retard pur (T) de 0,7 seconde** (latence système), une période d'échantillonnage de **0,07 seconde**, et un **gain statique (K =3,4)** calculé sur le régime établi, traduisant le rapport de conversion final entre l'angle de pitch commandé et la vitesse longitudinale obtenue.



Réponse normalisée sur Matlab avec tfest

3) Calcul du PID

Le calcul du régulateur PID dans votre script repose sur une séquence logique de trois opérations mathématiques distinctes, chacune associée à une instruction MATLAB précise pour transformer le modèle théorique en code de vol exploitable. Tout d'abord, la dynamique identifiée est recalée sur la physique réelle du drone en multipliant le système par son gain statique mesuré et en lui injectant le retard pur de transmission, ce qui correspond aux lignes `plant = sys * staticgain` et `plant.inputdelay = pitchdelay` ; cette étape est critique car elle interdit au futur correcteur d'ignorer la latence de 0,7 seconde qui rendrait le drone instable. Ensuite, l'optimisation proprement dite est confiée à la fonction `[PIDpitchspeed, info] = pidtune(plant, 'PID')`, qui résout les équations de la boucle fermée pour trouver l'unique combinaison de gains `K_p`, `K_i`, `K_d` offrant la marge de phase suffisante pour la

stabilité. Enfin, parce que le calculateur du drone ne traite pas des fonctions continues mais des échantillons temporels, le modèle compensé est mathématiquement transformé en équations de récurrence (domaine de Z) via la commande plantz = c2d(plant, Ts), figeant ainsi la loi de commande pour la cadence de 70 ms du processeur.



Les paramètres K du PID

B) Modélisation du roll

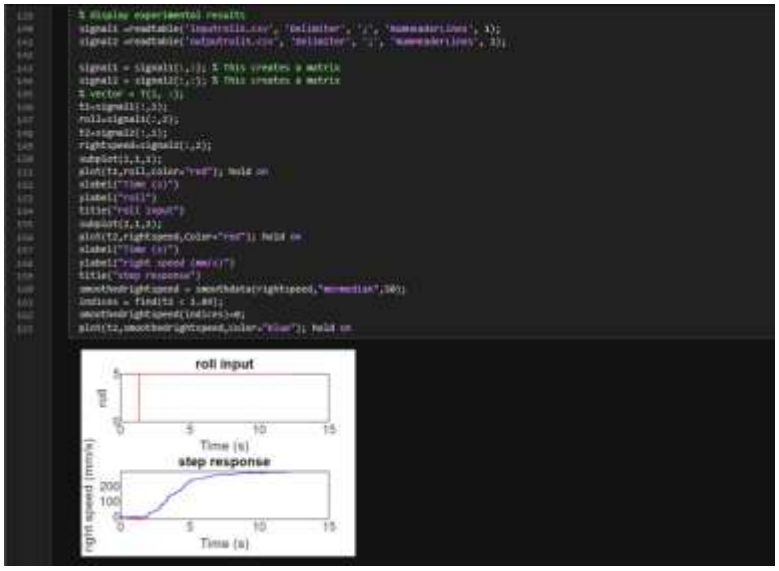
1) Identification

L'étude du roll s'inscrit dans la continuité de l'identification du pitch, mais elle caractérise la capacité du drone à se déplacer latéralement en réponse à une inclinaison commandée . Dans le contexte de l'hyperréalité, la maîtrise de cet axe est critique pour assurer des trajectoires précises et compenser les dérives éventuelles. L'objectif est ici de déterminer la fonction de transfert **$H_{roll}(p)$,** qui traduit mathématiquement le rapport entre la vitesse latérale obtenue et l'angle de roulis imposé.

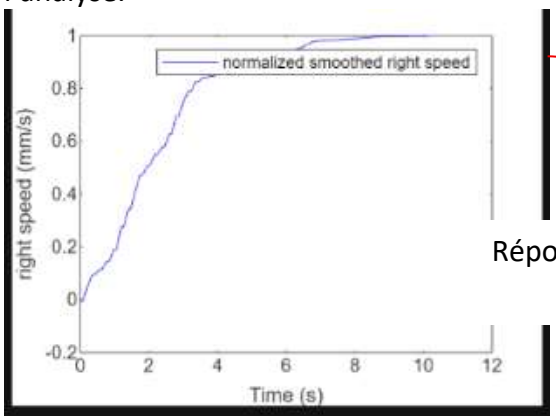
Fondamentalement, la dynamique de translation latérale s'apparente à un système du premier ordre. En effet, lorsque le drone s'incline, la projection du vecteur poussée crée une force latérale constante qui accélère le drone, tandis que la traînée aérodynamique s'oppose progressivement au mouvement jusqu'à atteindre une vitesse limite. Cette physique se formalise par l'équation canonique suivante :

$$Hroll(p) = \frac{K}{1 + \tau rollp} e^{-TP}$$

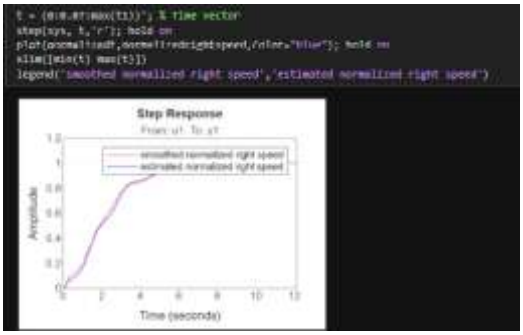
Cette modélisation repose sur l'identification de trois paramètres structurels spécifiques à l'axe latéral. Le **Gain Statique (K_roll)** définit la sensibilité du système : il correspond au rapport entre la vitesse latérale stabilisée et l'angle d'inclinaison maintenu (exprimé en (m/s)/deg). La **Constante de Temps (τ_roll)** quantifie l'inertie latérale de l'appareil, c'est-à-dire la rapidité avec laquelle il atteint sa vitesse de croisière. Enfin, le **Retard Pur (T_d)** modélise la latence incompressible de la chaîne de transmission et de la motorisation.
Pour les besoins de simulation avancée dans le Jumeau Numérique, nous adoptons, comme pour le pitch , une stratégie d'identification dite "Boîte Noire" via l'algorithme tfest de MATLAB. Cette méthode permet de s'affranchir de la simplicité théorique pour capturer des dynamiques d'ordre supérieur (modes vibratoires, asymétries aérodynamiques gauche/droite). Enfin, le contrôleur de vol opérant en temps discret, cette fonction de transfert continue est systématiquement transposée dans le domaine de Z (H (z)) selon la période d'échantillonnage T_s = 0.07s$, rendant le modèle mathématique directement exploitable par le microcontrôleur du drone anafi.



L'essai indiciel pour le roulis a consisté à envoyer une commande d'angle latérale passant brusquement de **0 à 5 unités** à l'instant **t = 1,4 seconde**. L'analyse immédiate des données brutes (outputroll5.csv) montre que le drone n'était pas parfaitement immobile au départ, présentant un **offset négatif important d'environ -10,5 unités**. Comme pour le pitch, l'approche "MATLAB" impose de neutraliser ce biais (suppression de la composante continue) pour ne pas fausser l'analyse.



Réponse normaliser



L'identification du modèle roll commence par la correction du s initial de -10,45 mesuré avant l'essai. La variation réelle de vitesse se calcule en soustrayant ce biais à la valeur finale stabilisée de 295,68, soit une amplitude corrigée de 295,68 - (-10,45) = 306,13. Le gain statique du système est alors déterminé par la division de cette réponse par la commande d'entrée de 5, ce qui donne K = 306,13 / 5 = 61,23. La constante de temps est ensuite isolée en relevant l'instant où la réponse atteint 63% de son évolution (t = 4,34s) et en lui soustrayant l'instant du déclenchement de l'échelon (t = 1,40s), soit Tau = 4,34 - 1,40 = 2,94s. Ces résultats numériques valident la fonction de transfert utilisée pour le réglage du correcteur, en y intégrant le retard de transmission fixe de 0,7s.

2) Calcul du PID

Le calcul du régulateur PID pour le roll repose sur la même séquence logique de trois opérations mathématiques pour transformer le modèle identifié en code de vol. Tout d'abord, la dynamique latérale est recalée sur la physique réelle en intégrant le gain statique calculé de 61,23 et le retard pur de transmission de 0,7 seconde, une étape critique pour compenser la très forte sensibilité de cet axe par rapport au pitch. Ensuite, l'optimisation est confiée à la fonction algorithmique qui détermine l'unique combinaison de **gains proportionnel, intégral et dérivé capable** de stabiliser ce système vif sans provoquer de dépassement dangereux. Enfin, le modèle compensé est mathématiquement transformé en équations de récurrence via la conversion en temps discret avec une période de 0,07 seconde, figeant ainsi la loi de commande adaptée à la fréquence de calcul du processeur embarqué du drone.

```
disp(PIDrollspeed);

  pid with properties:

            Kp: 0.0239
            Ki: 0.0108
            Kd: 0.0049
            Tf: 0
      IFormula: ''
      DFormula: ''
    InputDelay: 0
   OutputDelay: 0
     InputName: {''}
     InputUnit: {''}
    InputGroup: [1×1 struct]
    OutputName: {''}
    OutputUnit: {''}
   OutputGroup: [1×1 struct]
         Notes: [0×1 string]
      UserData: []
          Name: ''
            Ts: 0
      TimeUnit: 'seconds'
  SamplingGrid: [1×1 struct]
```

```
% Design a PID controller
% Use the 'pidtune' function to automatically tune the PID gains
[PIDrollspeed, info] = pidtune(plant, 'PID');

% Display the tuned PID controller
disp('Tuned PID roll Controller:');

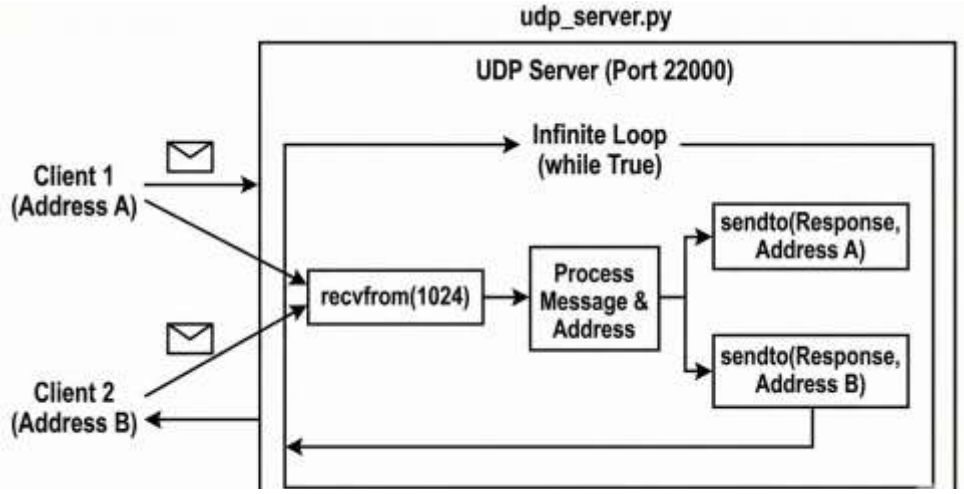
Tuned PID roll Controller:

disp(PIDrollspeed);
```

V) Protocole UDP et TCP/IP

A) Le Serveur UDP

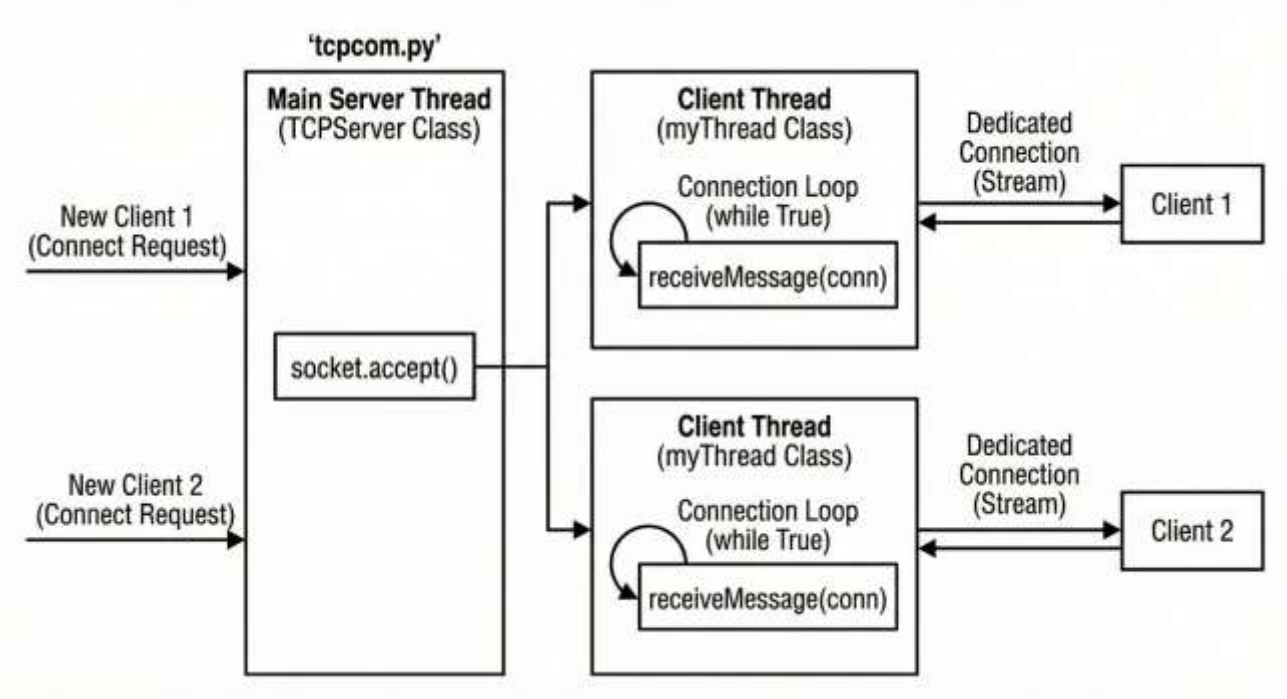
Le script udp_server.py met en œuvre un serveur basé sur le protocole UDP (User Datagram Protocol), caractérisé par son absence de connexion préalable. Comme illustré dans le schéma ci-dessous, ce serveur fonctionne selon un modèle "Fire and Forget" comparable à une boîte aux lettres. Le socket est configuré en mode Datagramme (SOCK_DGRAM) et le serveur entre dans une boucle infinie d'écoute passive sur le port 22000.



Le fonctionnement est intrinsèquement séquentiel et sans état. La méthode recvfrom(1024) bloque l'exécution jusqu'à l'arrivée d'un paquet provenant de n'importe quel client. À ce moment, le serveur capture simultanément le message et l'adresse de l'expéditeur, traite le message, et renvoie immédiatement une réponse à cette adresse précise via sendto(). Une fois la réponse envoyée, le serveur "oublie" ce client et retourne à l'état d'écoute pour le prochain paquet, quel que soit son expéditeur. Cette architecture est simple et légère, mais elle ne garantit ni l'ordre d'arrivée des paquets, ni leur réception, et traite les requêtes les unes après les autres, sans maintenir de session continue.

B) Le Serveur TCP

À l'opposé, le script tcpcom.py propose une architecture sophistiquée utilisant le protocole TCP (Transmission Control Protocol), qui est orienté connexion et garantit la fiabilité des échanges. Pour gérer plusieurs clients simultanément sans blocage, ce serveur adopte une stratégie de parallélisation par multithreading, comme le montre le schéma suivant.



Le rôle du serveur est divisé entre un thread principal (la classe TCPServer) et des threads ouvriers (myThread). Le socket est configuré en mode flux continu (SOCK_STREAM). Le thread principal agit comme un superviseur : il se met en écoute (listen) et utilise la méthode bloquante accept() pour attendre les nouvelles demandes de connexion. Lorsqu'un client se connecte, accept() renvoie un nouveau socket dédié à cette connexion unique. Le serveur principal instancie alors un nouvel objet myThread auquel il confie ce socket dédié, puis il retourne immédiatement à l'écoute d'autres clients. Chaque thread ouvrier gère ensuite de manière autonome et parallèle le dialogue

continu avec son client respectif via la méthode receiveMessage, garantissant une communication fiable et non bloquante pour l'ensemble du système.

VI) Proposition et idées d'amélioration de l'hyperréalité

A) Intégration de la Physique Virtuelle dans le Monde Réel

L'amélioration la plus spectaculaire consisterait à rendre les obstacles virtuels physiquement tangibles pour le drone. Actuellement, le jumeau numérique suit le drone ; l'idée serait d'inverser ce rapport de force pour que l'environnement virtuel contraigne le vol réel. Concrètement, si l'opérateur fait apparaître un mur invisible ou un cube virtuel sur l'interface 3D, le drone réel devra se heurter à cette frontière immatérielle comme si elle existait physiquement. Pour réaliser cela, il faudrait implémenter un algorithme de "Champs de Potentiels Artificiels" dans la boucle de contrôle. Le logiciel calculerait en temps réel une force de répulsion virtuelle dès que les coordonnées GPS ou optiques du drone s'approchent de l'objet 3D, modifiant la consigne envoyée au PID pour freiner ou dévier l'appareil avant l'impact virtuel, créant ainsi une véritable fusion des deux mondes.

B) Immersion et Retour de Force

Pour dépasser la simple visualisation sur écran, il serait pertinent d'ajouter un retour sensoriel à l'opérateur, fermant ainsi la boucle d'interaction homme-machine. L'objectif est de permettre au pilote de "sentir" les perturbations subies par le drone, comme les rafales de vent ou l'inertie lors des virages serrés. Cela pourrait se faire en connectant les données télémétriques (accéléromètres et gyroscopes reçus via UDP) à un joystick à retour de force ou à une veste haptique. Techniquement, il s'agirait de mapper les valeurs d'accélération latérale et verticale du drone sur les actionneurs du contrôleur de jeu via une interface DirectX ou une librairie dédiée en Python, offrant une téléprésence kinesthésique immédiate.

C) Compensation Prédictive par Intelligence Artificielle

Enfin, pour s'affranchir du délai critique de 0,7 seconde identifié lors de la modélisation, l'avenir du projet réside dans l'abandon du PID classique au profit d'une commande prédictive (MPC - Model Predictive Control) assistée par IA. Plutôt que de réagir à l'erreur passée, le système utiliserait un réseau de neurones entraîné sur le modèle "Boîte Noire" que vous avez identifié pour prédire le comportement futur du drone sur un horizon de quelques secondes. Ce "cerveau" numérique enverrait les commandes en avance de phase, annulant virtuellement la latence de transmission. Cela demanderait de développer un script Python utilisant des bibliothèques comme TensorFlow ou PyTorch, qui tournerait en parallèle du serveur TCP pour ajuster dynamiquement les gains ou les consignes avant même que l'opérateur ne perçoive le retard.

VII) Conclusion

En conclusion, ce projet a permis de valider concrètement le concept d'hyperréalité en établissant une symbiose fonctionnelle entre le drone physique et son jumeau numérique. La démarche s'est

articulée autour d'une identification rigoureuse des dynamiques de vol, nécessitant un pré-traitement méthodique des données expérimentales pour éliminer les biais statiques et isoler les réponses indicielles des axes de tangage et de roulis. L'exploitation de ces mesures via des modèles mathématiques de type boîte noire d'ordre élevé a rendu possible la synthèse de correcteurs PID numériques précis, prenant en compte les contraintes physiques réelles telles que les retards de transmission et la discrétisation temporelle indispensable au processeur embarqué. Parallèlement, l'analyse de l'architecture informatique sous-jacente a mis en lumière la complémentarité des protocoles réseaux, distinguant l'efficacité du mode datagramme UDP pour la fluidité des échanges de la robustesse du mode connecté TCP pour la supervision. Cette fusion entre l'automatique avancée et l'informatique industrielle aboutit finalement à un environnement de test sécurisé où le virtuel ne se contente plus de simuler la réalité, mais la pilote et la contraint activement.